RealtorDedupe — Step by Step Walkthrough

Overview

RealtorDedupe is an ML pipeline that identifies duplicate real estate agent records across multiple MLS systems and produces a clean, unique agent list.

Problem: The same agent appears under different names, phone formats, license prefixes, and emails across MLS systems. No single unique identifier exists.

Solution: A 5-step pipeline combining normalization, blocking, embeddings, and probabilistic matching.

Result: 99.7% F1 Score · 100% Precision · 0 False Positives

Tech Stack

Tool	Purpose
Python 3.x	Core language
Pandas	Data manipulation
sentence-transformers	Embedding model
Splink	Probabilistic record linkage
scikit-learn	Cosine similarity
MLflow	Experiment tracking (future)

Project Structure

```
RealtorDedupe/
├── data/
|   ├── raw/                        ← input data
|   ├── processed/                  ← intermediate outputs
|   └── output/                     ← final results
├── src/
```

```
|   ├── 01_agent_test_dataset.py     ← generate test data
|   ├── 02_normalize.py              ← clean & standardize
|   ├── 03_blocking.py               ← generate candidate pairs
|   ├── 04_embeddings.py             ← semantic similarity scoring
|   ├── 05_splink_match.py           ← probabilistic matching
|   └── 06_evaluate.py               ← evaluate & build unique list
├── config/
|   └── settings.py                  ← shared configuration
└── requirements.txt
```

Step 1 — Generate Test Dataset

File: `src/01_agent_test_dataset.py` **Output:** `data/raw/agent_test_dataset.csv`

Generates 505 synthetic agent records covering 9 test cases:

Test Case	Description	Purpose
TC1	Name variations	Thomas / Tom / Tommy / T.
TC2	Multi-state agents	Same agent in NC, CA, TX
TC3	Phone number changed	Same agent, different phone
TC4	Email variations	Same agent, different domains
TC5	License prefix variations	NC-12345 vs NC12345 vs 12345
TC6	Similar names, different agents	John Smith × multiple people
TC7	Exact duplicates	Easiest case to catch
TC8	Missing fields	Null phones, emails, licenses
TC9	Truly different agents	True negatives

Key field: `true_agent_id` — ground truth for evaluation.

```bash
python src/01_agent_test_dataset.py
```

Step 2 — Normalize

File: `src/02_normalize.py` **Input:** `data/raw/agent_test_dataset.csv` **Output:** `data/processed/normalized_agents.csv`

Standardizes all fields so downstream comparison is reliable.

Normalizations Applied

Phone numbers:

```
(704) 555-1234  →  7045551234
704.555.1234    →  7045551234
+17045551234    →  7045551234
```

License numbers:

```
NC-12345        →  12345
NC12345         →  12345
LIC-NC-12345    →  12345
```

Agent names:

```
"  THOMAS  JOHNSON  "  →  "thomas johnson"
"Thomas  Johnson"      →  "thomas johnson"
```

Emails:

```
"TJohnson@Gmail.COM"   →  "tjohnson@gmail.com"
```

New columns added

- `norm_phone`, `norm_license`, `norm_name`, `norm_email`, `norm_office`
- `first_name`, `last_name`, `email_username`, `email_domain`

```bash
python src/02_normalize.py
```

Step 3 — Blocking

File: `src/03_blocking.py` **Input:** `data/processed/normalized_agents.csv` **Output:** `data/processed/candidate_pairs.csv`

Groups records into buckets and generates candidate pairs within each bucket.

Why Blocking?

Without blocking: **127,260 comparisons** (505 × 504 ÷ 2) With blocking: **1,169 comparisons** — 99.1% reduction

Blocking Rules (OR logic)

Rule	Key generated	Example
Name prefix	first 3 chars last + first char first	`joh_t`
Phone last 7	last 7 digits of norm_phone	`5551234`
Email username	part before @	`tjohnson`
License number	normalized digits	`12345`

Records sharing **any one key** end up in the same block and are compared.

Output

- `record_id_1`, `record_id_2` — the pair
- `true_agent_id_1`, `true_agent_id_2` — ground truth
- `is_true_match` — 1 if same agent, 0 if different

```bash
python src/03_blocking.py
```

Step 4 — Embeddings

File: `src/04_embeddings.py` **Input:** normalized agents + candidate pairs **Output:** `data/processed/scored_pairs.csv`

Converts each agent record to a semantic vector and calculates cosine similarity for each candidate pair.

How Embeddings Work

Each agent record is combined into one text string:

```
"name: thomas johnson phone: 7045551234 email: tjohnson@gmail.com license: 12345
state: nc"
```

The embedding model (all-MiniLM-L6-v2) converts this to a vector of 384 numbers that captures semantic meaning.

Why Better Than String Comparison

```
String comparison:  "Thomas" vs "Tom"  →  43%  ❌
Embedding model:    "Thomas" vs "Tom"  →  94%  ✅
```

Cosine Similarity

Compares two vectors and returns a score from 0 to 1:

- `1.0` = identical
- `0.9+` = very similar (likely same agent)
- `0.5` = somewhat similar
- `0.0` = completely different

Results

- Auto Merge (≥ 0.95): 206 pairs
- LLM Review (0.75–0.95): 594 pairs
- Keep Separate (< 0.75): 369 pairs

```bash
python src/04_embeddings.py
```

Step 5 — Splink Matching

File: `src/05_splink_match.py` **Input:** normalized agents + scored pairs **Output:** `data/output/final_matches.csv`

Uses Splink to calculate match probabilities by weighing evidence against coincidence probability.

How Splink Works

Splink asks: *"Is this similarity a real match or just a coincidence?"*

```
License match  →  Very rare by coincidence  →  High weight
Phone match    →  Rare by coincidence       →  Medium weight
Name match     →  Common (John Smith)       →  Lower weight
```

Fields configured:

- `norm_name` — Jaro-Winkler at 0.95, 0.88, 0.70 thresholds
- `norm_phone` — Exact match
- `norm_email` — Exact match
- `norm_license` — Exact match
- `email_username` — Jaro-Winkler at 0.95, 0.88 thresholds

Final Score

```
final_score = (embedding_score × 0.40) + (splink_score × 0.60)
```

Splink weighted higher because it weighs evidence more carefully.

Results

- Auto Merge (≥ 0.95): 225 pairs — 100% precision
- LLM Review (0.75–0.95): 95 pairs — needs review
- Keep Separate (< 0.75): 849 pairs — 2 true matches missed

```bash
python src/05_splink_match.py
```

Step 6 — Evaluate & Build Unique Agent List

File: `src/06_evaluate.py` **Input:** normalized agents + final matches **Output:** `data/output/unique_agents.csv` + `data/output/evaluation_report.csv`

Evaluates pipeline performance and builds the final deduplicated agent list.

Evaluation Metrics

Metric	Definition	Result
Precision	Of matches found, how many correct?	100%
Recall	Of true matches, how many found?	99.4%
F1 Score	Balance of precision and recall	99.7%

Error Analysis

False Positives (wrong merges): 0 — we never merged different agents.

False Negatives (missed matches): 2

```
Jennifer Smith  vs  J. Smith   →  score: 0.35  (too different)
Michael Allen   vs  M. Allen   →  score: 0.33  (too different)
```

These are hard edge cases where only initial + last name with different data across MLS systems.

Building the Unique Agent List

Uses union-find algorithm to group all AUTO_MERGE pairs into connected components. The most complete record (fewest nulls) becomes the canonical record for each group.

```
Input:   505 raw records
Output:  359 unique agents
Removed: 146 duplicates (28.9% dedup rate)
```

```bash
python src/06_evaluate.py
```

Running the Full Pipeline

```bash
# Activate virtual environment
.\realtordedupe_env\Scripts\activate

# Run all steps in order
python src/01_agent_test_dataset.py
python src/02_normalize.py
python src/03_blocking.py
python src/04_embeddings.py
python src/05_splink_match.py
python src/06_evaluate.py
```

Configuration

All settings in `config/settings.py`:

```python
# Thresholds
HIGH_CONFIDENCE = 0.95   # auto merge above this
LOW_CONFIDENCE  = 0.75   # keep separate below this
                          # LLM review in between

# Model
EMBEDDING_MODEL = "all-MiniLM-L6-v2"
```

Known Limitations & Next Steps

Current limitations

- Initial-only name variations (J. Smith) can be missed
- Test dataset is synthetic — real data may have different patterns
- LLM review step not yet implemented

Next steps

1. Add LLM review using Claude API for the 95 ambiguous pairs
2. Test with real production data sample
3. Tune thresholds based on real data patterns
4. Move to Databricks + Zingg for million+ record scale
5. Build monitoring for production pipeline drift

Built as a prototype to validate the ML approach before production scaling on Databricks.